ZenoDEX: Full-System Architecture and Public Assurance Boundary

Dana Edwards

April 2026

Abstract

ZenoDEX is intended to be a proof-carrying exchange architecture, not only a spot DEX runtime. The whole system spans theorem-bearing spot kernels, fail-closed guard planes, certificate-carrying settlement and price surfaces, bounded stablecoin and perpetual-risk layers, narrow replayable runtime adapters, and a public assurance boundary that is deliberately narrower than the full target architecture. This short paper states that whole-system thesis and makes the current public claim boundary explicit.

Contributions. This paper contributes:

1. a compact whole-system decomposition;
2. a public-safe evidence discipline;
3. a guarantee-and-assumption summary for the main surfaces;
4. an explicit separation between the full architecture and the current RC1 release claim.

1 Whole-system thesis

The intended architecture is

$$\mathsf{ZenoDEX} := \mathsf{Exec} \wedge \mathsf{Guards} \wedge \mathsf{Cert} \wedge \mathsf{Control} \wedge \mathsf{Oracle} \wedge \mathsf{zUSD} \wedge \mathsf{Perps} \wedge \mathsf{Runtime} \wedge \mathsf{Assure}.$$

Standard reading: the whole system includes execution, guards, certificates, risk layers, runtime adapters, and assurance surfaces. Practical consequence: the correct review object is the architecture, not one isolated module.

The current public discipline is

$$\mathsf{PublicClaim} \subset \mathsf{EvidenceFrontier} \subset \mathsf{FullTargetArchitecture}.$$

Standard reading: the current release claim is narrower than the strongest public evidence, and both are narrower than the intended full system. Practical consequence: an honest paper may describe the full stack while still keeping the release claim conservative.

2 Evidence rule

The governing evidence order is

$$\texttt{proved} > \texttt{contract} > \texttt{implemented} > \texttt{replay\_backed} > \texttt{hypothesis}.$$

Standard reading: every claim is limited by the strongest public replayable support actually present. Practical consequence: theorem-bearing spot slices, packet contracts, and release gates are intentionally not collapsed into one generic label.

3 Execution and certificate planes

The canonical execution pattern is

$$\mathsf{Winner}(C) := \arg\min_{c \in C} \mathsf{Key}(c),$$

with certificate-bearing settlement shaped by

$$\mathsf{SettlementOK} \rightarrow \mathsf{CanonicalDeltaTable} \wedge \mathsf{ReplayVerifiable}.$$

Standard reading: bounded candidate families are resolved by explicit total keys, and accepted settlement must carry replay-checkable canonical meaning. Practical consequence: determinism is treated as semantic canonicality plus replayable packet structure, not only as stable iteration order.

The strongest public theorem stock today lives in the spot execution plane:

- canonical batch winner theorems,
- exact-in winner and packet shells,
- CPMM invariant and accounting theorems,
- fee-dust carry and anti-fragmentation helper theorems.

The certificate plane then lifts these local mechanisms into public packet and settlement surfaces.

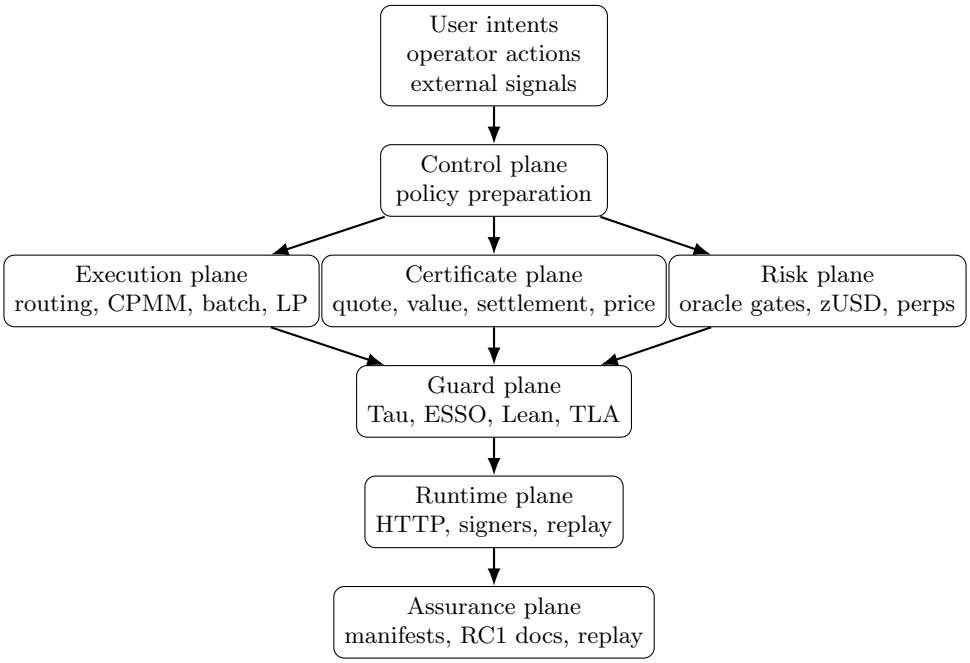

Figure 1: Whole-system stack. Advisory or planning surfaces may influence preparation, but execution authority flows only through fail-closed guards and narrow runtime adapters.

4 Control, oracle, and runtime planes

The full system is not only math plus packets. The admission layer is

$$\mathsf{AdmittedIntent} := \mathsf{NormalFormOK} \wedge \mathsf{SignatureOrWitnessOK} \wedge \mathsf{NonceOK} \wedge \mathsf{BoundaryContractOK},$$

and the price-admission layer is

$$\mathsf{PriceInputAccepted} \rightarrow \mathsf{FreshnessBound} \wedge \mathsf{PendingStateConsistent} \wedge \mathsf{ProvenancePacketCoherent}.$$

Standard reading: the runtime only becomes authoritative after normalization, authentication, nonce discipline, and oracle/provenance checks succeed. Practical consequence: the public system claim must include ingress and price discipline, not only the inner execution kernels.

The supported runtime path is still narrower than the whole repository:

$$\mathsf{SupportedHTTP} \subset \mathsf{AllRuntimeEndpoints}.$$

Practical consequence: the paper describes the whole runtime stack, but the release claim stays bounded to the documented supported boundary.

5 Risk and runtime planes

The runtime path is narrower than the full repository:

$$\mathsf{SupportedHTTP} \subset \mathsf{AllRuntimeEndpoints}.$$

The monetary and derivatives layers are also narrower than the end-state target. Representative public formulas are:

$$\mathsf{zUSDAdmissible} \rightarrow \mathsf{DebtConservation} \wedge \mathsf{CollateralConservation},$$

$$\mathsf{PerpTransitionAllowed} \rightarrow \mathsf{FreshOracleWindow} \wedge \mathsf{BoundedMoveGate}.$$

Standard reading: zUSD and perps are covered by explicit bounded public safety claims, not by a blanket claim that the whole risk stack is fully proved. Practical consequence: the full system is covered in the architecture, but the public release claim remains more conservative than the target design.

6 Assurance plane

The public assurance plane is executable:

$$\mathsf{PublishedClaimOK} \rightarrow \mathsf{ManifestPinned} \wedge \mathsf{ReplaySurfaceTracked} \wedge \mathsf{GeneratedEvidenceRebuildable}.$$

Standard reading: the release claim is only meaningful if the public replay lanes and manifests remain synchronized. Practical consequence: the repo's assurance documents are part of the system, not only documentation about it.

Surface	Intended guarantee	Current strongest public evidence	Main limit
Spot execution	canonical bounded winner relation	proved	bounded emitted candidate domain
Settlement / quote binding	replayable canonical packet surfaces	contract + tests + proved helper shells	packet semantics still depend on explicit admission contracts
Runtime ingress	signer / nonce / receipt discipline	contract + tests	supported HTTP subset is intentionally narrow
zUSD / oracle safety	bounded solvency and stale-oracle fail-closed posture	mixed	not every stablecoin claim is release-backed
Perps / funding / liquidation	bounded risk-envelope posture	mixed	not a blanket derivatives-complete proof
RC1 public release	replayable public assurance surface	contract + replay evidence	clean tree and honest exclusions required

Table 1: Compact guarantee matrix.

7 RC1 public boundary

The current public RC1 contract is

$$\mathsf{RC1ClaimOK} := \mathsf{CleanTree} \wedge \mathsf{ScopeFrozen} \wedge \mathsf{ReplayGreen} \wedge \mathsf{ExclusionsHonest}.$$

Standard reading: the RC1 claim is configuration-specific, replay-backed, and explicitly bounded. Practical consequence: the public release claim is narrower than the whole architecture described in this paper.

8 End-state proof agenda

The remaining agenda is to widen bounded canonicality into broader packet and generator surfaces, strengthen zUSD and perps from mixed public posture toward stronger theorem-backed claims, and keep runtime replay boundaries aligned with the same canonical payload semantics enforced below them.

This paper is grounded only in public-safe artifacts:

- `docs/PUBLIC_ASSURANCE_REPLAY.md`
- `docs/RC1_SCOPE.md`
- `docs/RC1_VERIFIED_SURFACE_MATRIX.md`
- `docs/TLA_CLAIM_SUMMARY.md`
- `docs/claims_registry.yaml`
- theorem-bearing files under `lean-mathlib/Proofs/`
- public kernels under `src/kernels/dex/`